



Figure 1: Susceptible, infected and recovered populations in a SIR model

```
b = 4 # Contact ratio is 2
t_array = np.arange(0, 100, 1)
RES = integrate.odeint(dif_eq, (S0,I0,R0), t_array,args=(b,k))
graph(RES)
```

You define a method *dif_eq* which computes the derivatives for the system of differential equations. The parameters to this method are an array containing the current values, the current time and additional arguments b and k , in this case.

The key method is *odeint* whose parameters are a function defining the differential equations—for example, the *dif_eq* above, a list containing the initial values, a time array at which solutions need to be found, and any additional arguments to the *dif_eq* function. The best way to observe the results is graphically.

```
import matplotlib.pyplot as plt
def graph(RES):
    plt.subplot(2,1,1)
    plt.plot(t_array, RES[:,0], '-', label='Susceptible')
    plt.plot(t_array, RES[:,2], '-', label='Recovered')
    plt.xlabel('Days')
    plt.ylabel('Susceptible/Recovered')
    plt.legend()
    plt.subplot(2,1,2)
    plt.plot(t_array, RES[:,1], '-', label='Infected')
    plt.xlabel('Days')
    plt.ylabel('Infected')
    plt.legend()
    plt.show()
```

It is at times easier to show sub-plots. In the example above, a plot with two rows and one column is created. In the first row, a sub-plot shows the susceptible and recovered populations. The second sub-plot shows the infected population. The result is shown in Figure 1. The infections peak about 60 days after the outbreak and the

maximum people infected at one time is about 16 percent, which is 1,60,000 in a population of a million!

Impact of vaccinations and quarantine

Vaccination results in a reduction of the susceptible population. These people directly move into the recovered category without being infected. You can modify the *dif_eq* method as follows:

```
def dif_eq_vac(V,b,k,vac):
    dVdt = np.zeros(3)
    dVdt[0] = -b*V[0]*V[1] - vac
    dVdt[1] = b*V[0]*V[1] - k*V[1]
    dVdt[2] = k*V[1] + vac
    return dVdt
```

If 1,000 or 5,000 people are vaccinated each day, you will need to modify your code to include:

```
t_array = np.arange(0, 200, 1)
RES1 = integrate.odeint(dif_eq, (S0,I0,R0), t_array,args=(b,k))
RES2 = integrate.odeint(dif_eq_vac, (S0,I0,R0), t_array,args=(b,k,1000/P))
RES3 = integrate.odeint(dif_eq_vac, (S0,I0,R0), t_array,args=(b,k,5000/P))
```

Let's suppose a fraction q of the infected are quarantined. Hence, only $(1-q)$ fraction will be a part of the infected group. The remaining will be treated as recovered/removed. The code for the differential equations becomes, as follows:

```
def dif_eq_quarantine(V,b,k,q):
    dVdt = np.zeros(3)
    dVdt[0] = -b*V[0]*V[1]
    dVdt[1] = b*(1-q)*V[0]*V[1] - k*V[1]
    dVdt[2] = k*V[1] + b*q*V[0]*V[1]
    return dVdt
```

Let's assume that a third of the infected population is quarantined. Add the code and call the plot routine:

```
RES4 = integrate.odeint(dif_eq_quarantine, (S0,I0,R0), t_array,args=(b,k,.333))
graph2(RES1, RES2, RES3, RES4)
```

The modified graph routine shows the infected populations, with and without vaccination:

```
def graph2(RES1, RES2, RES3, RES4):
    plt.plot(t_array, RES1[:,1], '-', label=None)
    plt.plot(t_array, RES2[:,1], '-', label='1000 per day')
    plt.plot(t_array, RES3[:,1], '-', label='5000 per day')
    plt.plot(t_array, RES4[:,1], '-', label='1/3 Quarantined')
    plt.xlabel('Days')
    plt.ylabel('Infected')
    plt.legend()
    plt.show()
```